

Emnekode: PG3401
Emnenavn: Programming in C for Linux

Individuell hjemmeeksamen
candidate number: 307



Semester: Våren 2026

Denne oppgaven er gjennomført som en del av utdannelsen ved Høgskolen Kristiania.
Høgskolen er ikke ansvarlig for oppgavens metoder, resultater,
konklusjoner eller anbefalinger.

Contents

Task 1. Theory.....	2
Explain what the C programming language can be used for	2
Who is Richard Stallman.....	2
Explain what the sudo command is used for on a Linux-based system	3
Task 2. File management and functions	4
Task 3. List handling.....	5
Task 4. Threads	7
Task 5. Network	9
Task 6. Problem solving	10
References:	13

Task 1. Theory

Explain what the C programming language can be used for

C is a general-purpose, procedural programming language. Originally developed by Dennis Ritchie at Bell Laboratories between 1969 and 1973 (Kernighan & Ritchie, 1988). Despite its age, C remains one of the most acknowledged programming languages in the world, and it has consistently ranked among the top languages in the industry (TIOBE, 2025). C is perhaps best known as the language in which operating systems are written. The Unix operating system was rewritten in C in 1973, demonstrating that high-level language could be used for systems programming without sacrificing performance. When it comes to embedded systems, C is the dominant language used for systems development, where programs run on hardware with severely limited memory and processing power (Barr & Massa, 2006). Many programming languages are themselves implemented in C. The reference implementations of Python, Perl and Ruby are all written in C. This reflects C's role as a kind of universal assembly language, close enough to the hardware to be efficient, but portable enough to run across architectures (Kernighan & Ritchie, 1988).

Network software, including core internet infrastructure tools, is commonly written in C. Programs such as OpenSSL, the OpenSSH client and the Nginx web server are implemented in C. In the field of cybersecurity, C is used extensively for exploit development, vulnerability research and writing low-level security tools, because it allows direct manipulations of memory, which is central to understanding buffer overflows and other memory corruption vulnerabilities (Seacord, 2013).

C also plays a central role in the development of compilers and other language tooling. C has also historically been the language of choice for real-time systems and safety-critical applications, including avionics, automotive systems and industrial automation. Standards such as MISRA C were developed specifically to constrain the use of C in safety embedded environments, providing coding guidelines that reduce the risk of undefined behavior and improve software reliability (MISRA, 2013).

Who is Richard Stallman

Richard Stallman (born 1953) is an American software developer and activist he is widely regarded as one of the most influential figures in the history of free and open-source software. He is most known as the founder of the GNU-Project and the Free Software Foundation (FSF). In 1983, Stallman launched the GNU Project with the goal of creating a completely free Unix-like operating system (Store Norske Leksikon, 2024). The name "GNU" is recursive acronym standing for "GNU's Not Unix", which reflecting both its Unix compatibility and its independence.

In 1985, Stallman published the *GNU Manifesto*, articulating his philosophy that software should be freely available for anyone to use, study, modify and redistribute (Free Software Foundation, 2015). This philosophy became the basis for the concept of free software, where "free" refers to freedom, not price. To protect these freedoms legally, Stallman developed the GNU General Public License (GPL), one of the most widely used software licenses in the world, which ensures that derivative works remain free and open (GNU Operating System, 2025).

Stallman's work laid the ideological and technical groundwork for the later Linux-based GNU/Linux operating system, which combined the Linux kernel developed by Linus Torvalds in 1991 with the GNU userland tools. Stallman has consistently argued that the combined system should be referred to as "GNU/Linux" to acknowledge the GNU project's contribution (Store Norske Leksikon, 2024).

Beyond his technical contributions, Stallman has been a significant figure in the legal and political discourse surrounding intellectual property and software patents. He has been a vocalist in the media about software patents, arguing that they represent a fundamental threat to software development by allowing trivial or broad algorithmic concepts to be monopolized (Stallman, 2004).

Explain what the sudo command is used for on a Linux-based system

sudo (short for "superuser DO") is a command-line utility on Unix-like operating systems that allows a permitted user to execute a command with the privileges of another user, typically the root (superuser) account. It is a fundamental tool in Linux system administration and security. On Linux systems, the root account has unrestricted access to all files, processes and system configuration. Logging in directly as root is considered a security risk, as any mistake or malicious action would have system-wide consequences. A typical usage example would be installing a software package: "sudo apt install gcc -y". Without sudo, this command would be denied for a normal user, as installing system packages requires root privileges.

Which users are permitted to use sudo, and for which commands, is defined in the "/etc/sudoers" file. Most modern Linux distributors such as Ubuntu, the first user created during installation, are automatically added to the sudo group, granting them full administrative privileges. Another important aspect of sudo is its logging and auditing capability. Every command executed through sudo is recorded in the system log, typically via syslog or journald, capturing the invoking user's identity. The command executed and a timestamp (Nemeth, 2010).

In cybersecurity and penetration testing, misconfigured sudo rules are a common privilege escalation vector. If a user is permitted to run certain commands as root without a password, or if a permitted command can spawn a shell, an attacker may leverage this to gain full root access.

Task 2. File management and functions

- EWA starting task2

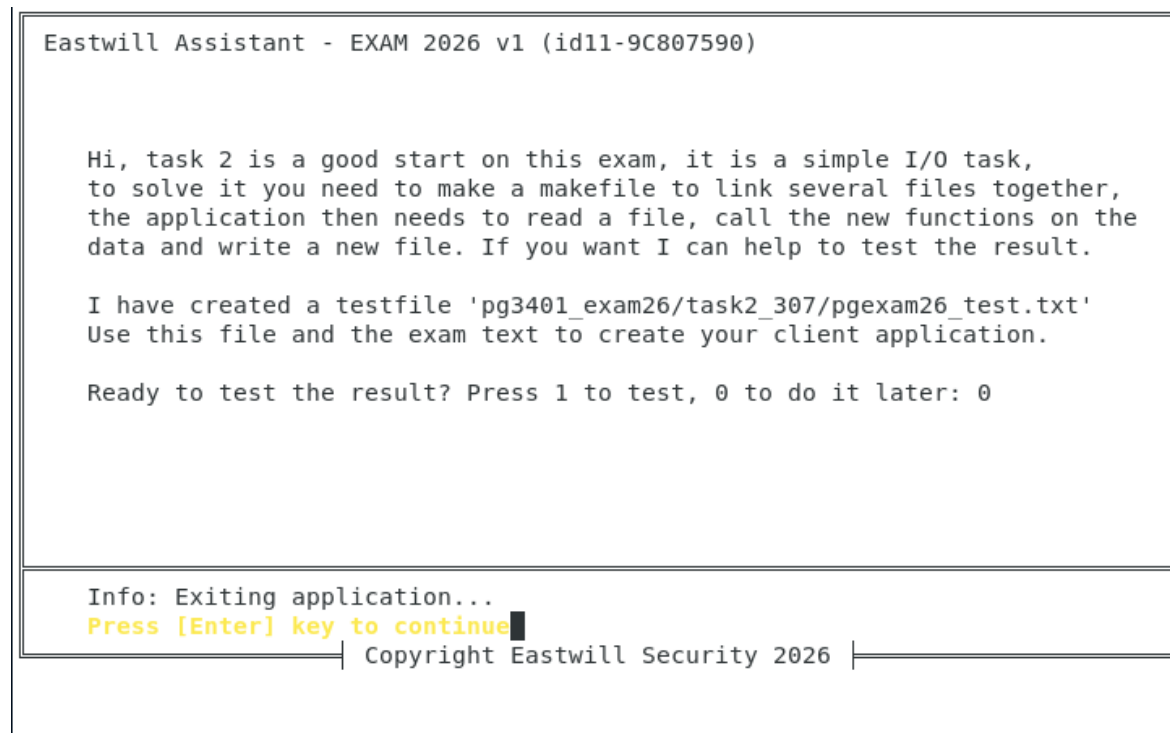


Figure 1

- Terminal output after running my program

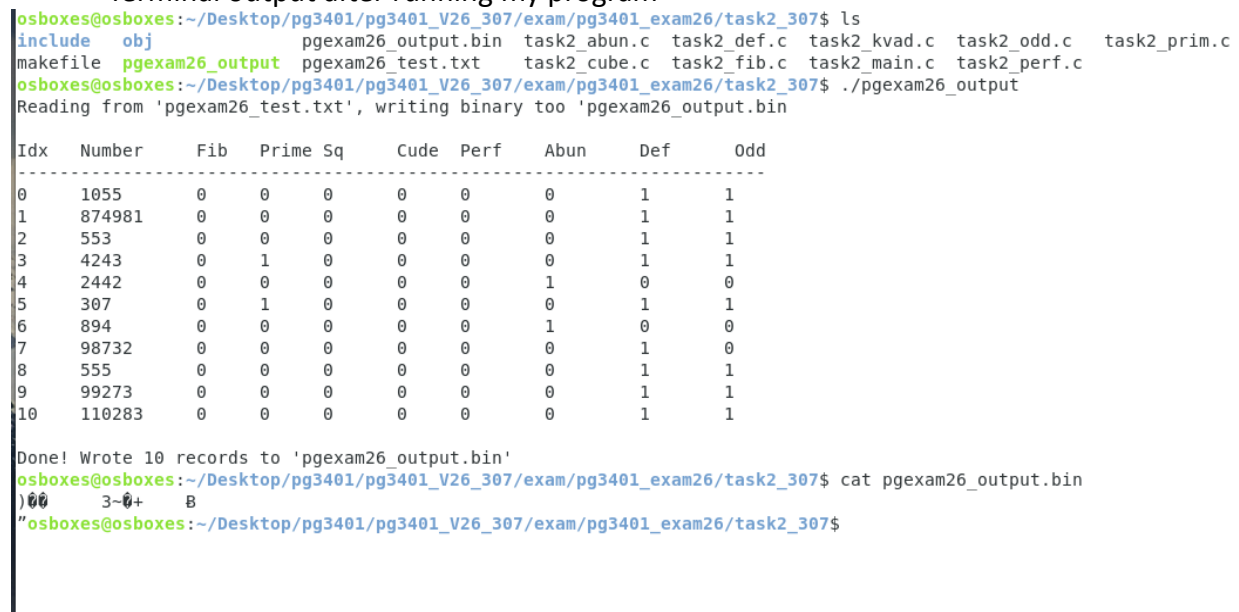


Figure 2

- EWA checking accepting task2

```
Eastwill Assistant - EXAM 2026 v1 (id13-9C809890)

You have asked me to check your result, let us have a look...

Successfully opened 'pg3401_exam26/task2_307/pgexam26_output.bin'.
File is 440 bytes large, let me read it.
Successfully read the binary file - check
I have checked some of the numbers - check

I have not checked the code quality, that I will leave up to the
assessor, but I can read the file and it looks fine.
When you think your code quality is good enough this task is done.
Good work on this task - keep it up :-)
```

Press [Enter] key to continue

Copyright Eastwill Security 2026

Figure 3

The objective of this task was to develop a C program that reads a sequence of integers from a text file, evaluates each integer against a set of mathematical classification functions, and writes the results to a binary output file using a predefined struct. The program was structured across multiple source files following a modular pattern in C development. A header file (task2.h) was created in a dedicated “include/” directory, containing function declarations for all eight classification functions provided by the EWA tool, as well as the (TASK2_NUMBERS_METADATA) struct definition. A “Makefile” based on the course lecture template was used to automate compilation, linking all object files and the math library(-lm) into the final executable. The program opens (pgexam26_test.txt) using fopen() in reading mode and parses each integer on separate lines using fscanf(). For each integer read, the program calls the eight EWA-provided functions. The return values, of type bool, are cast to int and stored in the corresponding fields of a (TASK2_NUMBERS_METADATA) struct instance. The iIndex field is assigned a zero-based sequence number, incrementing for each record processed. The struct is written to (pgexa26_output.bin) in binary mode using fwrite(), writing one struct per call. With 11 integers in the input file and a struct size of 40 bytes, the resulting binary file is 440 bytes. This approach ensures a compact, fixed-size binary format suitable for direct memory mapping and machine reading.

Task 3. List handling

- Running my program

```

osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task3_307$ ./task3_grading
--- Exam Setup ---
Number of tasks (1-10): 2
Max points for task 1: 10
Max points for task 2: 10
Exam configured: 2 task(s)

---- Exam Grading System ----
1. Add candidate manually
2. Load candidates from file
3. Grade a task for a candidate
4. Print all candidates with grades
5. Print candidates by grade
6. print ungraded / incomplete candidates
7. Print full report for one candidate
0. Exit
-----
Choose an option: 2
Enter filename: candidates.txt
Candidate 10042 added
Candidate 10043 added
Candidate 10044 added
Candidate 10045 added
Candidate 10046 added
Loaded 5 candidates from candidates.txt

---- Exam Grading System ----
1. Add candidate manually
2. Load candidates from file
3. Grade a task for a candidate
4. Print all candidates with grades
5. Print candidates by grade
6. print ungraded / incomplete candidates
7. Print full report for one candidate
0. Exit
-----

```

Figure 4

```

-----
Choose an option: 3
Enter candidate ID: 10042
Task number (1-2): 2
Points awarded: 10
Justification: God besvarelse
Task2 graded for candidate 10042: 10 pts

---- Exam Grading System ----
1. Add candidate manually
2. Load candidates from file
3. Grade a task for a candidate
4. Print all candidates with grades
5. Print candidates by grade
6. print ungraded / incomplete candidates
7. Print full report for one candidate
0. Exit
-----
Choose an option: 4

--- All Candidates ---
CandidateID      Score    MaxPts    Grade
-----
10042             18        20        A
10043              0        20        F
10044             10        20        D
10045              0        20        F
10046              0        20        F
-----
Total candidates: 5

```

Figure 5

The applications support the following operations, all accessible through an interactive menu:

- Add candidate manually, allocates a new Candidate node via malloc, initializes all fields, and appends it to the tail of the list

- Load candidates from file which reads candidate IDs line from a plain text file, skipping comment lines (#), and calls addCandidate for each valid entry
- Grade a task that locates a candidate by ID, validates the task number and points against the allowed range, sets the points and justifications and then recalculates the total score
- Print all candidates with grades this traverses the full list and prints each candidate's ID, total score, maximum possible score and calculated letter grade
- Print ungraded/incomplete candidates, this prints all the candidates where one or more tasks have not yet been graded
- Print full report for one candidate. Prints score and justification for every task of a specific candidate, along with their final grade

Task 4. Threads

- EWA verifying the changes to the file task4_threads.c and part 1 of task4

```
Eastwill Assistant - EXAM 2026 v1 (id13-9C801890)

You have asked me to check your result, let us have a look...
Successfully opened 'pg3401_exam26/task4_307/task4_threads.c' - check
Changes detected in file (hash 88f115da)
Successfully read the source file - check
Successfully found a makefile in the task4 folder - check
Successfully changed fopen call - check
Successfully found mutex use - check
Successfully found explicit init - check
Successfully found 16 comments - check

Part I approved, press [Enter] to get Part II

Press [Enter] key to continue
Copyright Eastwill Security 2026
```

Figure 6

- Running the updated task4_threads.c and decrypting the file for part 2, showing the prime numbers, which number was the key, and the content of the task4_codes.bin file

```

osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task4_307$ ./task4_threads task4_primes.txt
--- Running task4 ---
checking for prime numbers...
Prime found: 1000000007
Prime found: 1000000021
Prime found: 1000000009
Key found: 1000000009
Decrypted: BENGt: I think candidate 0307 is doing well so far! :-)
Prime found: 1000000033
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task4_307$ cat task4_plain.hash
4294960866
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task4_307$

```

Figure 7

- Also running Valgrind for checking memory leaks (valgrind --leak-check=yes --track-origins=yes ./task4_threads task4_primes.txt)

```

==3455== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==3455== Command: ./task4_threads task4_primes.txt
==3455==
--- Running task4 ---
checking for prime numbers...
Prime found: 1000000021
Prime found: 1000000007
Prime found: 1000000009
Key found: 1000000009
Decrypted: BENGt: I think candidate 0307 is doing well so far! :-)
Prime found: 1000000033
==3455==
==3455== HEAP SUMMARY:
==3455==    in use at exit: 0 bytes in 0 blocks
==3455== total heap usage: 20 allocs, 20 frees, 20,504 bytes allocated
==3455==
==3455== All heap blocks were freed -- no leaks are possible
==3455==
==3455== For counts of detected and suppressed errors, rerun with: -v
==3455== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Figure 8

Part 1: Prime Number Search

The program reads a list of candidate numbers from a file passed as a command-line argument. Two POSIX threads are created, sharing a THREADARGS struct. A mutex protects file reads so only one thread reads at a time, and a second mutex protects writes to the shared linked list of prime numbers found.

[Figure 6: EWA accepting Part 1]

Part 2: XTEA Decryption

Each prime number found is tested as an XTEA key against the encrypted file (Task4_code.bin). The file is decrypted block by block using the provided tean function. If the first 5 bytes of the decryption result match "BENGt", the key is correct. A third mutex signals both threads to stop searching. The decrypted content is saved to (task4_plain.txt) and an IP checksum to (task4_plain.hash).

[Figure 7: Showing program running and output of the two decrypted files]

Task 5. Network

- Terminal output after running the program and connecting with EWA

```
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task5_307$ ./task5 -port 2626 -id Smtptest
Starting EWP server: id=Smtptest port=2626
Listening on 127.0.0.1:2626
Waiting for EWA connection...
Client connected: 127.0.0.1
Sent: 220 server accept
Received: HELO
Sent: 250 HELO response
Received: MAIL FROM: <bengt@test.com>
Sent: 250 Sender address ok
Received: RCPT TO: <student@kristiania.no>
Sent: 250 Recipient address ok
Received: DATA command, filename: to_33-9C808590.eml
Sent: 354 Ready for message
DEBUG FILE: iFilePayload=12 iFileBody=12
DEBUG: iPayloadSize=64 iHeaderSize=8 iBodySize=56
File to 33-9C808590.eml saved (12 bytes)
Sent: 250 File received ok
Received: QUIT
Sent: 221 Goodbye
Client disconnected
^C
```

Figure 9

- The output from EWA after connecting to the right server and port

```
Eastwill Assistant - EXAM 2026 v1 (id33-9C804290)

Successfully connected to 127.0.0.1, port 2626
Successfully received: EWP0064|220 127.0.0.1 SMTP Smtptest 19:05:24
Successfully sent: EWP0064|HELO bengt.127.0.0.1
Successfully received: EWP0064|250 127.0.0.1 Hello bengt
Successfully sent: EWP0064|MAIL FROM: <bengt@test.com>
Successfully received: EWP0064|250 Sender address ok
Successfully sent: EWP0064|RCPT TO: <student@kristiania.no>
Successfully received: EWP0064|250 Recipient address ok
#1 Successfully sent: EWP0064|DATA to_33-9C804290.eml
#1 Successfully received: EWP0064|354 ready for message
#1 Successfully sent: EWP0012|Test .
#1 Successfully received: EWP0064|250 File received ok
Successfully sent: EWP0064|QUIT
Successfully received: EWP0064|221 Goodbye

Press [Enter] key to continue
Copyright Eastwill Security 2026
```

Figure 10

- Checking the file data

```
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task5_307$ file to_33-9C808590.eml
to_33-9C808590.eml: data
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task5_307$ cat -A to_33-9C808590.eml
M-^D,M--M-{-^@^@^@M-0qMM-aosboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task5_307$
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task5_307$
```

Figure 11

Task5 required implementing a TCP that communicated with the EWA client using a custom SMTP-inspired protocol. The server accepted a port number and server ID from the command line and processed a fixed sequence of protocol commands defined in the header file (ewpdef.h). Upon receiving a connection from EWA, the server processed a fixed sequence of protocol commands. Each incoming message was read using a receive loop that endured all expected bytes were received before parsing, handling the case where TCP may deliver data in fragments.

The implementation used blocking I/O on a single thread, processing one command at a time in sequence. Once the full exchange was completed successfully, EWA confirmed that the correct responses had been sent and accepted the task

Task 6. Problem solving

```
Eastwill Assistant - EXAM 2026 v1 (id33-9C809590)

Successfully sent packet 0 19338
Successfully received ACK packet for : 0 19338
Successfully sent packet 19338 19239
Error: received ACK Number 0, expected same as last sequence number 19338.
Successfully sent packet 57916 18938
Error: received ACK Number 0, expected same as last sequence number 57916
ERROR: send failed with 104 when sending packet 38577 19339
WARNING: rcv got 0 of expected 20 bytesame as last sequence number 38577

Press [Enter] key to continue
Copyright Eastwill Security 2026
```

Figure 12

```

DEBUG rawsum=64488
DEBUG raw: seq=2861957120 iszeOfPacket=19256 checksum=12923
Out of sequence: got 2861957120 expected 1, sending NACK
Sent ACK: ack=1 flags=0x20 bytes=21
DEBUG before swap: sizeOfPacket raw bytes = 38 4b
62 a4 ec 5e aa 96 00 00 00 00 00 00 05 01 38 4b 7b 32 00 00
DEBUG: iDataSize=19256
Recv: seq=38570 flags=0x01 size=19256 pData=(nil)
DEBUG rawsum=64488
DEBUG raw: seq=2861957120 iszeOfPacket=19256 checksum=12923
Out of sequence: got 2861957120 expected 1, sending NACK
Sent ACK: ack=1 flags=0x20 bytes=21
DEBUG before swap: sizeOfPacket raw bytes = 38 4b
62 a4 ec 5e aa 96 00 00 00 00 00 00 05 01 38 4b 7b 32 00 00
DEBUG: iDataSize=19256
Recv: seq=38570 flags=0x01 size=19256 pData=(nil)
DEBUG rawsum=64488
DEBUG raw: seq=2861957120 iszeOfPacket=19256 checksum=12923
Out of sequence: got 2861957120 expected 1, sending NACK
Sent ACK: ack=1 flags=0x20 bytes=21
Error reading header
File saved: butterfly.bmp (19250 bytes)
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task6_307$ tree
.
├── butterfly.bmp
├── include
│   └── ewpdef.h
├── makefile
├── obj
│   └── task6_main.o
├── task6
└── task6_main.c

2 directories, 6 files
osboxes@osboxes:~/Desktop/pg3401/pg3401_V26_307/exam/pg3401_exam26/task6_307$

```

Figure 13

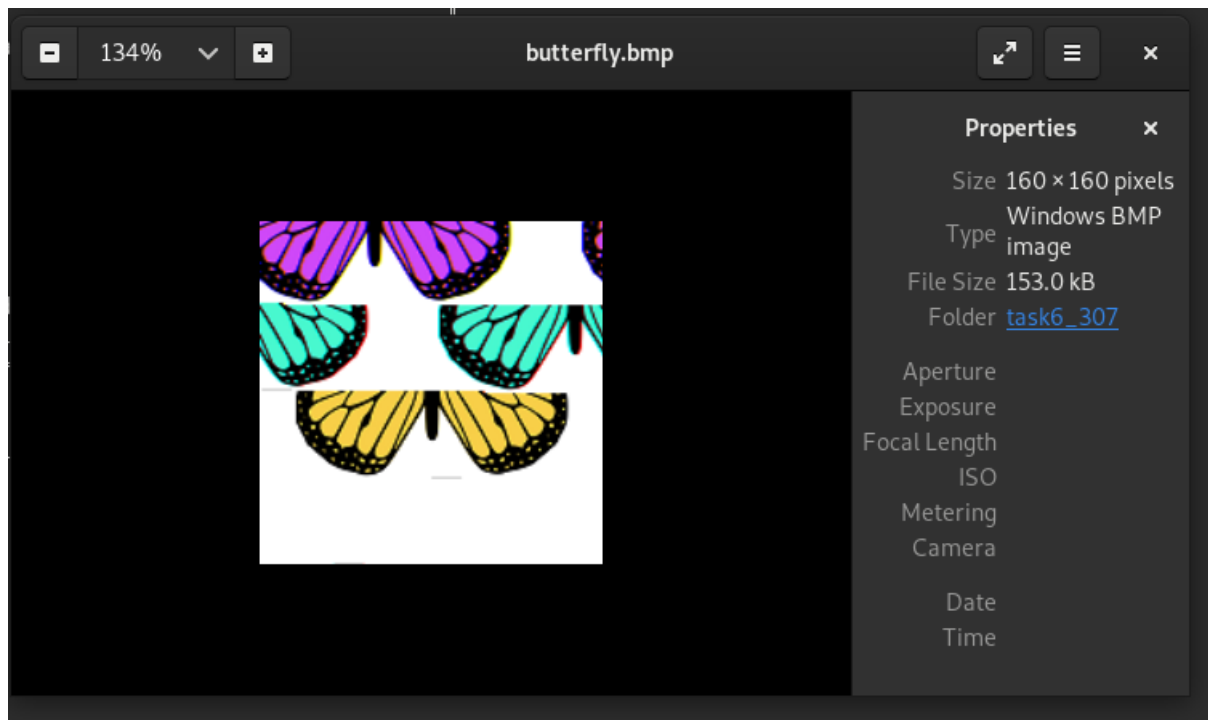


Figure 14

Task 6 required implementing an EWP-over-TCP client capable of receiving a file transmitted in sequenced packets, verifying each packet and acknowledging receipt to the server. The first challenge encountered was byte order handling. The protocol header fields are transmitted in network byte order. This required explicit byte swapping of `uiSequenceNumber`, `uiAckNumber` and `usSizeOfPacket`, I ended up making a manual `swapU32` function.

The second and most persistent challenge was checksum verification. Multiple algorithms were attempted, including a simple byte sum. Despite these attempts, the computed checksum consistently failed to match the value transmitted by the server. The root cause was not fully isolated within the time available and checksum verification was ultimately disabled to allow the rest of the protocol logic to be validated.

The third challenge was the ACK number expected by the server. EWA expected the ACK number to reflect the next expected byte offset, that is `sequence number + payload size`, rather than simply echoing the sequence number of the received packet.

Once this was corrected, the first packet was acknowledged successfully.

Despite these issues, the client successfully received and reconstructed the transmitted bitmap image, which was saved to disk as `butterfly.bmp` and verified visually to be correct.

References:

Barr, M., & Massa, A. (2006). *Programming embedded systems* (2nd ed.). O'Reilly Media.

Free Software Foundation. (2025). *About the Free Software Foundation*.

<https://www.fsf.org/about/>

GNU Operating System. (2025). The GNU General Public License.

<https://www.gnu.org/licenses/gpl-3.0.html>

Kernighan, B. W., & Ritchie, D. M. (1998). *The C programming language* (2nd ed.). Prentice Hall

MISRA. (2013). MISRA C: 2012. *Guidelines for the use of the C language in critical systems*. MIRA Limited.

Nemeth, E., Snyder, G., Hein, T. R., & Whaley, B. (2010). *Unix and Linux system administration handbook* (4th ed.). Prentice Hall

Stallman, R. M. (2004). *Software patents. Obstacles to software development*. Free Software Foundation. <https://www.gnu.org/philosophy/software-patents.html>

Store Norske Leksion. (2024). *GNU – operativsystem*.

[https://snl.no/GNU - operativsystem](https://snl.no/GNU_-_operativsystem)

Seacord, R. C. (2013). *Secure coding in C and C++* (2nd ed.). Addison-Wesley.

https://www.sei.cmu.edu/documents/1312/2005_009_001_52710.pdf